

# Reverse proxy

## Auteur

Nom : [DEMAISON Camille]

---

Dans le cadre du projet nous devons déployer sur un **docker** un reverse proxy qui permettra de faire tampon pour les sites web exposé sur l'IP publique.

Dans le cahier des charges il est demandé de :

1. Passer toutes les connections de `itway.fr` au serveur `dmz-web.int.itway.fr` et de `webmail.itway.fr` au serveur `mail.itway.fr`.
2. Tout développer en respectant les bonnes pratiques en termes de sécurité. Exposé au danger des attaques externes il faudra anticiper les vecteurs de failles. La mise en place d'un certificat SSL signé par l'autorité de certification interne pour sécuriser les échanges avec les serveurs et clients est nécessaire.
3. Le tout doit être déployé par ansible et doit fonctionner automatiquement.

Nginx nous est paru comme une évidence par sa simplicité de mise en place et par le fait que l'on ai déjà utilisé sur d'autres composantes du projet. En effet il permet d'allier simplicité de mise en place et sécurité.

### Structure du projet :

Actuellement l'application est situé dans un dossier nommé `/opt/flask_app/` avec cette structure :

```
/etc/nginx/sites-available/    # Pour configurer Nginx
└─ dmz-rproxy.conf             # Configuration de notre reverse-proxy
  en HTTP/HTTPS

/etc/nginx/sites-/             # Pour configurer Nginx
└─ reverse-proxy               # Configuration de notre reverse-proxy en
  HTTP/HTTPS
```

## Technologies utilisées :

| Technologie | Usage                                                      |
|-------------|------------------------------------------------------------|
| Nginx       | Gestion de la connexion chiffrée avec certificat de la PKI |

## Installation

Pour l'installer il suffit d'entrer ceci :

```
apt install nginx -y
```

## Configuration

Dans un premier temps il faudra vérifier que les **DNS** de l'AD est de la DMZ sont bien configuré :

- **AD** => Renvoie l'IP interne du reverse, ici `172.16.1.5` car accès direct.
- **DMZ** => Renvoie l'IP publique, ici `192.168.122.150` car utilisé par les utilisateurs pour se connecter. (règle de NAT faite par le Stormshield)

Ensuite pour configurer le reverse proxy on a du créer un fichier de configuration situé dans `/etc/nginx/sites-available/`.

Voici la composition de `dmz-rproxy.conf` :

```
# Redirige les url spécifiques HTTP vers HTTPS
server {
    listen 80;
    server_name itway.fr webmail.itway.fr;
    return 301 https://$host$request_uri;
}
```

```

# Redirige les noms d'hostes http incorrects vers itway.fr en https
server {
    listen 80 default_server;
    return 301 https://itway.fr$request_uri;
}

# Proxy HTTPS pour le site principal (WordPress sur DMZ-WEB)
server {
    listen 443 ssl http2;
    server_name itway.fr;

    ssl_certificate /etc/ssl/certs/dmz.cert.pem;
    ssl_certificate_key /etc/ssl/private/dmz.key.pem;
    ssl_trusted_certificate /usr/local/share/ca-certificates/myCA.crt;

    include /etc/nginx/snippets/ssl-params-web.conf;

    access_log /var/log/nginx/itway.access.log;
    error_log /var/log/nginx/itway.error.log;

    location / {
        # Nom d'hôte interne du serveur WordPress
        proxy_pass https://172.16.1.2; #dmz-web.int.itway.fr; a voir
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Connection "";

        # Permet de valider les certificats SSL
        proxy_ssl_verify off;
        # CA interne
        proxy_ssl_trusted_certificate /usr/local/share/ca-
certificates/myCA.crt;
    }
}

# Proxy HTTPS pour la messagerie web
server {
    listen 443 ssl http2;
    server_name webmail.itway.fr;

    ssl_certificate /etc/ssl/certs/dmz.cert.pem;
    ssl_certificate_key /etc/ssl/private/dmz.key.pem;
    ssl_trusted_certificate /usr/local/share/ca-certificates/myCA.crt;

    include /etc/nginx/snippets/ssl-params-mail.conf;

```

```

access_log /var/log/nginx/webmail.access.log;
error_log /var/log/nginx/webmail.error.log;

location / {
    # Nom d'hôte interne du serveur Webmail
    proxy_pass https://172.16.1.4;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header Connection "";

    # Permet de valider les certificats SSL
    proxy_ssl_verify off;
    # CA interne
    proxy_ssl_trusted_certificate /usr/local/share/ca-
certificates/myCA.crt;
}
}

```

Nous avons ici 4 configurations qui permettent de :

- Rediriger les requêtes HTTP pour nos serveurs configurés vers la version HTTPS de la requête.
- Définir la route HTTP par défaut, elle est utilisée lorsque l'URL ne correspond à aucune autre route définie. Ici elle renvoie automatiquement une requête HTTPS pour notre site WEB vitrine (redirection).
- Définir la route principale pour le site vitrine. Ici tout ce qui arrive sur `itway.fr` va être transféré à notre serveur web `https://172.16.1.2`.
- Définir la route principale pour le site de la boîte mail. Ici tout ce qui arrive sur `webmail.itway.fr` va être transféré à notre serveur web `https://172.16.1.4`.

La configuration est similaire à celle faite pour l'INTRANET, la seule grande différence ici que comme il est exposé au WEB nous avons dû ajouter des mesures de sécurité en plus.

Deux fichiers de configurations de la sécurité ont été ajoutés :

- `/etc/nginx/snippets/ssl-params-web.conf`
- `/etc/nginx/snippets/ssl-params-mail.conf`

Voici son contenu :

```
# Autorise uniquement les versions de TLS sécurisées
ssl_protocols TLSv1.2 TLSv1.3;

# Donne la priorité aux suites de chiffrement choisies par le serveur
ssl_prefer_server_ciphers on;

# Liste des suites de chiffrement autorisées, compatibles TLS 1.3
ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-RSA-AES128-GCM-SHA256';

# Durée de validité des sessions TLS (améliore les performances pour les connexions récurrentes)
ssl_session_timeout 1d;

# Cache partagé des sessions TLS, pour éviter de renégocier à chaque fois
ssl_session_cache shared:SSL:50m;

# Désactive les tickets TLS (risques de compromission de session si volés)
ssl_session_tickets off;

# Utilisation d'un fichier Diffie-Hellman pour améliorer la sécurité des échanges
# Ce fichier doit être généré une fois avec : openssl dhparam -out /etc/ssl/certs/dhparam.pem 2048
ssl_dhparam /etc/ssl/certs/dhparam.pem;

# Active HSTS (HTTP Strict Transport Security) pour forcer le client à utiliser HTTPS uniquement
# max-age=15768000 → environ 6 mois
# includeSubDomains → s'applique aussi aux sous-domaines
add_header Strict-Transport-Security "max-age=15768000; includeSubDomains" always;

# Protection contre le Clickjacking (empêche le site d'être inclus dans une iframe)
add_header X-Frame-Options DENY;

# Empêche l'interprétation automatique du contenu comme un autre type (protection MIME)
add_header X-Content-Type-Options nosniff;

# Active la protection XSS sur les navigateurs compatibles
add_header X-XSS-Protection "1; mode=block";
```

```
# Cache les informations de version du serveur NGINX (protection contre les scans de version)
server_tokens off;
```

Cette configuration a été construite suite la recherche sur divers forum qui expliquaient les mesures minimales de sécurité.

La configuration est la même pour les deux fichier mais j'ai du désactiver cette ligne pour le serveur mail car elle empêchait les preview des mails (affichage dans une iframe).

```
# Protection contre le Clickjacking (empêche le site d'être inclus dans une iframe)
add_header X-Frame-Options DENY;
```

Pour utiliser le reverse proxy il suffit d'activer et lancer la configuration précédente avec :

```
ln -s /etc/nginx/sites-available/dmz-rproxy.conf /etc/nginx/sites-enabled/nginx
nginx -t && nginx -s reload # Car pas de systemd
```

### Info

Comme pour l'INTRANET il faut faire attention a bien supprimer la conf par défaut : ça permet d'éviter d'être parasité.

En effet la config par défaut écoute sur le port 80 et donc peut avoir des comportements inattendus.

Elle est à ces emplacements :

```
/etc/nginx/sites-available/default
/etc/nginx/sites-enabled/default
```

# Script de démarrage

Un script de démarrage automatique a du être fait pour que la machine se lance automatique ment au lancement.

Pour cela on a édité le script qui se lance démarrage : `/gns3/run-cmd.sh` :

```
#!/bin/sh
# run docker startup, first arg is new PATH, remainder is command

PATH="$1"
shift

echo "Starting services..."
# Démarrer BIND en arrière plan :
nohup /usr/sbin/named -4 -u bind -g > /var/log/named.log 2>&1 &
# Démarrer le serveur SSH :
service ssh start &
# Démarrer Wazuh
service wazuh-agent start &

# Chemin vers notre reverse proxy
NGINX_SITE_AVAILABLE="/etc/nginx/sites-available/dmz-rproxy.conf"
NGINX_SITE_ENABLED="/etc/nginx/sites-enabled/dmz-rproxy.conf"

# Configurer Nginx si nécessaire
if [ ! -f "$NGINX_SITE_ENABLED" ]; then
    echo "Lien symbolique Nginx absent, creation..."
    ln -s "$NGINX_SITE_AVAILABLE" "$NGINX_SITE_ENABLED"
else
    echo "Lien symbolique Nginx deja present."
fi

# Verifier la config Nginx
echo "Verification de la configuration Nginx..."
if nginx -t; then
    # Démarrer Nginx si pas démarre
    if ! pgrep nginx > /dev/null; then
        echo "Nginx non démarre, démarrage..."
        nginx
    fi
    echo "Reload de Nginx..."
    nginx -s reload
else
    echo "Erreur dans la configuration Nginx, abandon."
    exit 1
fi
```

```
fi

echo "=== Demarrage termine avec succes ==="

exec "$@"
```

J'ai récupéré et adapté le script utilisé pour l'INTRANET car il fonctionnait très bien.

## Déploiement via ansible

Pour automatiser le déploiement des fichiers de configuration via ansible ont donc été mis en place selon cette architecture :

```
reverse-proxy-nginx/
├─ playbook.yml
├─ roles/
│   └─ nginx_rproxy/
│       └─ files/
│           ├── dmz-rproxy.conf
│           ├── ssl-params-web.conf
│           └─ ssl-params-mail.conf
```

Nous avons le playbook principal qui contient ceci :

```
- name: Déploiement NGINX Reverse Proxy
  hosts: DMZ-PROXY
  become: true
  tasks:

    - name: Installer NGINX
      apt:
        name: nginx
        state: present
        update_cache: yes

    - name: Supprimer le site par défaut de NGINX s'il existe
      file:
        path: /etc/nginx/sites-enabled/default
        state: absent
```

- **name:** Supprimer le site par défaut de NGINX s'il existe  
**file:**
  - path:** /etc/nginx/sites-available/default
  - state:** absent
- **name:** Copier la configuration du reverse proxy  
**copy:**
  - src:** roles/nginx\_rproxy/files/dmz-rproxy.conf
  - dest:** /etc/nginx/sites-available/dmz-rproxy.conf
  - owner:** root
  - group:** root
  - mode:** '0644'
- **name:** Copier les snippets SSL (web)  
**copy:**
  - src:** roles/nginx\_rproxy/files/ssl-params-web.conf
  - dest:** /etc/nginx/snippets/ssl-params-web.conf
  - owner:** root
  - group:** root
  - mode:** '0644'
- **name:** Copier les snippets SSL (mail)  
**copy:**
  - src:** roles/nginx\_rproxy/files/ssl-params-mail.conf
  - dest:** /etc/nginx/snippets/ssl-params-mail.conf
  - owner:** root
  - group:** root
  - mode:** '0644'
- **name:** Générer le fichier dhparam.pem s'il n'existe pas  
**command:** openssl dhparam -out /etc/ssl/certs/dhparam.pem 2048  
**args:**
  - creates:** /etc/ssl/certs/dhparam.pem
- **name:** Créer le lien symbolique dans sites-enabled  
**file:**
  - src:** /etc/nginx/sites-available/dmz-rproxy.conf
  - dest:** /etc/nginx/sites-enabled/dmz-rproxy.conf
  - state:** link
  - force:** yes
- **name:** Vérifier la configuration NGINX  
**command:** nginx -t  
**register:** nginx\_check  
**failed\_when:** "'test is successful' not in nginx\_check.stderr"
- **name:** Recharger NGINX  
**command:** service nginx restart

```
- name: Vérifier le script de démarrage existant
  copy:
    src: roles/nginx_rproxy/files/run-cmd.sh
    dest: /gns3/run-cmd.sh
    mode: '0755'
    owner: root
    group: root
```

Pour lancer le déploiement de la configuration il faut ensuite entrer cette commande :

```
ansible-playbook -i hosts.ini srv-rproxy/rproxy-playbook.yml
```

### Info

Il ne faudra pas oublier de suivre la documentation déploiement d'un conteneur et d'ajouter les certificats nécessaires. (playbook certificat à lancer aussi)

```
sftp root@172.16.1.5
```

```
put /etc/myCA/certs/dmz.cert.pem /etc/ssl/certs/
```

```
put /etc/myCA/private/dmz.key.pem /etc/ssl/private/
```

**Si la machine a changé alors il faudra supprimer la clef SSH sur le machines qui auront besoin d'y accéder.**

Il suffit d'entrer cette commande :

```
ssh-copy-id root@IP_MACHINE_CIBLE
```

Puis d'entrer la commande demandé par le terminal :

```
ssh-keygen -f '/root/.ssh/known_hosts' -R 'IP_MACHINE_CIBLE'
```

# Tests et vérifications

Pour vérifier que tout est opérationnel on va devoir vérifier certains points :

- L'accès par réseau interne et externe est bien fonctionnel
- Les communications sont bien chiffrées
- La machine est opérationnelle automatiquement après le démarrage
- Le déploiement par ansible est bien effectif

Différents tests ont été procédés :

- Accès par réseau interne et externe :

Pour tester les pages on peut passer par un navigateur web ou utiliser la commande `wget`

```
wget https://itway.fr --no-check-certificate

[...]  
Resolving itway.fr (itway.fr)... 192.168.122.150  
Connecting to itway.fr (itway.fr)|192.168.122.150|:443... connected.  
[...]  
HTTP request sent, awaiting response... 200 OK  
[...]  
Saving to: 'index.html.1'
```

La page est téléchargée, on peut donc l'afficher avec l'outil `cat`, on peut donc constater que c'est bien la bonne.

On peut faire de même avec `https://webmail.intranet.fr` et constater le même résultat.

- Pour cela on a utilisé Wireshark et l'on a fait une capture au moment d'une requête sur <https://itway.fr> :

|               |              |              |     |                                                                |
|---------------|--------------|--------------|-----|----------------------------------------------------------------|
| 459 38.516606 | 192.168.20.2 | 172.16.1.5   | TCP | 54 62685 → 443 [FIN, ACK] Seq=3245 Ack=150605 Win=262656 Len=0 |
| 460 38.516894 | 172.16.1.5   | 192.168.20.2 | TCP | 54 443 → 62685 [FIN, ACK] Seq=150605 Ack=3246 Win=64128 Len=0  |
| 461 38.519556 | 192.168.20.2 | 172.16.1.5   | TCP | 54 62685 → 443 [ACK] Seq=3246 Ack=150606 Win=262656 Len=0      |

Si on analyse les flux on obtient :

```
#
000-00-0000000000#y10002+ajJ(000004I74uu000+)0-010000,0o20R(00g0XK0)000p00-L0rpf=>0z0K0Pa0rb [Jw00000v0x\@0p^z0000t000g=00"1000S000t
eA=0000ddoc000wg0U0"[C0=1000qEYg00[0n00o0Etc0]000000 000k000-0000Vnd00=0u0Na0t0lme10 00L0-00007H0900010m100+C0<x000VS0001000_<"0Q70,00
v0=0000.00000000[000(0000(D7000000003!{#0300'
v00007000-00000u000_00)0001y0t00fY
0001+0000xyR0p000[q00'000513j0K'000:1S(4dk0<0g;v00;00Q000007yL0:H0\v0v0602;+LiD0)Y
nz00000000000000000000000000[M]M0K
000V1Eq0(0(R0R)T)BH000t0)X00[0000070|1000_LK0k000w0000h000w\bl0Z008;000u0u000000UKk0dy0)0000009x;100ku0xF00000Q[0uQ^000ZX000S_010k0)0000000R1S_u0t000000000;10000'xt00F=000;t703'00p00000sp
0000
0E[("0wd8Ku0cH;200000;1E0y007040'L0700W
000000K0k00000S0_Ryc000000(U0000000,0y00000000000-00-y/00b\M00000000ez0ysx|Vn000B-0c0/万000000000c0j000y0 0v0u 00)00S00+J00Z7000=00J00-0r000k00d@sptG01rf00v0
0J70f000rk00(2Rd00[km010000z0
000"P/100000000z0yw0u0000L0Q0G000hFE0000000)0);s0S"000300<00v00*20Q000S0H0]00q0
0-g000,000104y00000It0(0_k_0z 000>S0u/00010VX10)"000
0COS
0
0uy0/h000000K0@"000'00=vMI0
")000FXZ0;000000k00000ks:xS(0RYZE0000v000050000000A+000H)000skw)01n0[w(B(\^
000000;0y00=000-[0L0v0E000)000*UC000R000ItQ0K 0:w00's/0/000
SIR000
```

On constate bien que la communication est **chiffré avec TLSv1.2** aussi bien d'un coté que de l'autre du reverse proxy.

- Lancement automatique du serveur :  
Pour le tester on a juste à stopper la machine et de la relancer.

En se connectant en SSH on constate bien que le script personnalisé est bien lancé que le site est de nouveau joignable. Le temps de lancement étant rapide c'est un succès.

- Déploiement via ansible :  
En suivant le protocole expliqué précédemment on obtient bien une machine fonctionnelle et qui contient bien tout les fichiers nécessaires.

En redémarrant la machine on constate aussi que le script est présent et fonctionnel :

```
Starting services...
Lien symbolique Nginx deja present.
Verification de la configuration Nginx...
wazuh-agent: unrecognized service
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
 * Starting OpenBSD Secure Shell server sshd
Nginx non démarre, démarrage...
Reload de Nginx...
2025/06/02 20:26:14 [notice] 27557#27557: signal process started
=== Démarrage termine avec succes ===
```

[ OK ]